```
<template>
    <view class="page">
        <view class="wrap">
            <!-- 头部时间天气 -->
            <view class="weather-section">
                <view class="weather-left">
                    <text class="weather-icon">{{weatherIcon}}</text>
                    <view>
                        <text class="weather-temp">{{weatherTemp}}&deg;C</text>
                        <text class="weather-desc">{{weatherDesc}}</text>
                    </view>
                </view>
                <view class="weather-right">
                    <text class="weather-time">{{clockTime}}</text>
                    <text class="weather-date">{{clockDate}}</text>
                    <text class="weather-location">{{weatherLoc}}</text>
                </view>
            </view>

            <!-- 设备信息 -->
            <view class="profile-section">
                <view class="profile-avatar">&#x1F50B;</view>
                <view class="profile-info">
                    <text class="profile-name">本质安全型防爆电源</text>
                    <text class="profile-desc">智能监控系统 &middot; 实时监测中
</text>
                </view>
                <view class="status-badge" :class="deviceOnline ? 'online' : 'offline'">
                    {{ deviceOnline ? '已连接' : '未连接' }}
                </view>
            </view>

            <!-- 数据网格 -->
            <view class="section-title">&#x1F4CA; 实时数据</view>
            <view class="data-container">

                <!-- 甲烷浓度 -->
                <view class="data-item">
                    <view class="data-left">
                        <text class="data-emoji">&#x1F525;</text>
                    </view>
                    <view class="data-right">
                        <text class="data-label">甲烷浓度</text>
                        <text class="data-value" :style="{color:
```

```
deviceColorMap.ch4_concent}">{{ formatVal(deviceValueMap.ch4_concent) }} <text class="data-unit">%</text></text>
                </view>
            </view>

            <!-- 一氧化碳浓度 -->
            <view class="data-item">
                <view class="data-left">
                    <text class="data-emoji">&#x1F4A8;</text>
                </view>
                <view class="data-right">
                    <text class="data-label">一氧化碳</text>
                    <text class="data-value" :style="{color: deviceColorMap.co}">{{ formatVal(deviceValueMap.co) }} <text class="data-unit">ppm</text></text>
                </view>
            </view>

            <!-- 二氧化碳浓度 -->
            <view class="data-item">
                <view class="data-left">
                    <text class="data-emoji">&#x1F33F;</text>
                </view>
                <view class="data-right">
                    <text class="data-label">二氧化碳</text>
                    <text class="data-value" :style="{color: deviceColorMap.co2}">{{ formatVal(deviceValueMap.co2) }} <text class="data-unit">ppm</text></text>
                </view>
            </view>

            <!-- PM2.5 -->
            <view class="data-item">
                <view class="data-left">
                    <text class="data-emoji">&#x1F637;</text>
                </view>
                <view class="data-right">
                    <text class="data-label">PM2.5</text>
                    <text class="data-value" :style="{color: deviceColorMap.pm2_5}">{{ formatVal(deviceValueMap.pm2_5) }} <text class="data-unit">&mu;g/m&sup3;</text></text>
                </view>
            </view>
```

```
                    <!-- 环境湿度 -->
                    <view class="data-item">
                        <view class="data-left">
                            <text class="data-emoji">&#x1F4A7;</text>
                        </view>
                        <view class="data-right">
                            <text class="data-label">环境湿度</text>
                            <text class="data-value" :style="{color: deviceColorMap.env_humidity}">{{ formatVal(deviceValueMap.env_humidity) }} <text class="data-unit">%RH</text></text>
                        </view>
                    </view>

                    <!-- 环境温度 -->
                    <view class="data-item">
                        <view class="data-left">
                            <text class="data-emoji">&#x1F321;</text>
                        </view>
                        <view class="data-right">
                            <text class="data-label">环境温度</text>
                            <text class="data-value" :style="{color: deviceColorMap.env_temp}">{{ formatVal(deviceValueMap.env_temp) }} <text class="data-unit">&deg;C</text></text>
                        </view>
                    </view>

                    <!-- 功率器件温度 -->
                    <view class="data-item">
                        <view class="data-left">
                            <text class="data-emoji">&#x1F50C;</text>
                        </view>
                        <view class="data-right">
                            <text class="data-label">器件温度</text>
                            <text class="data-value" :style="{color: deviceColorMap.device_temp}">{{ formatVal(deviceValueMap.device_temp) }} <text class="data-unit">&deg;C</text></text>
                        </view>
                    </view>

                    <!-- 输出电流 -->
                    <view class="data-item">
                        <view class="data-left">
                            <text class="data-emoji">&#x26A1;</text>
                        </view>
```

```
                        <view class="data-right">
                            <text class="data-label">输出电流</text>
                            <text class="data-value" :style="{color: deviceColorMap.output_current}">{{ formatVal(deviceValueMap.output_current) }} <text class="data-unit">A</text></text>
                        </view>
                    </view>

                    <!-- 输出电压 -->
                    <view class="data-item">
                        <view class="data-left">
                            <text class="data-emoji">&#x1F50B;</text>
                        </view>
                        <view class="data-right">
                            <text class="data-label">输出电压</text>
                            <text class="data-value" :style="{color: deviceColorMap.output_voltage}">{{ formatVal(deviceValueMap.output_voltage) }} <text class="data-unit">V</text></text>
                        </view>
                    </view>

                    <!-- 运行时长 -->
                    <view class="data-item">
                        <view class="data-left">
                            <text class="data-emoji">&#x23F1;</text>
                        </view>
                        <view class="data-right">
                            <text class="data-label">运行时长</text>
                            <text class="data-value" :style="{color: deviceColorMap.runtime_hours}">{{ formatVal(deviceValueMap.runtime_hours) }} <text class="data-unit">h</text></text>
                        </view>
                    </view>

                    <!-- 健康评分 -->
                    <view class="data-item">
                        <view class="data-left">
                            <text class="data-emoji">&#x1F49A;</text>
                        </view>
                        <view class="data-right">
                            <text class="data-label">健康评分</text>
                            <text class="data-value" :style="{color: deviceColorMap.health_score}">{{ formatVal(deviceValueMap.health_score) }} <text class="data-unit">分</text></text>
```

```
                    </view>
                </view>

                <!-- 占位（保持网格对齐） -->
                <view class="data-item data-item-invisible"></view>

            </view>

            <!-- 远程控制 -->
            <view class="control-section">
                <view class="section-title">&#x1F527; 远程控制</view>

                <view class="control-card">
                    <view class="control-info">
                        <text class="control-icon">&#x1F4A1;</text>
                        <text class="control-label">LED 灯</text>
                    </view>
                    <view class="control-btns">
                        <button class="ctrl-btn ctrl-on" :class="deviceValueMap.LED ? 'active' : ''" @click="setProperty('LED', true)" :disabled="settingInProgress === 'LED'"> 开 启 </button>
                        <button class="ctrl-btn ctrl-off" :class="!deviceValueMap.LED ? 'active' : ''" @click="setProperty('LED', false)" :disabled="settingInProgress === 'LED'"> 关 闭 </button>
                    </view>
                </view>

                <view class="control-card">
                    <view class="control-info">
                        <text class="control-icon">&#x1F514;</text>
                        <text class="control-label">蜂鸣器报警</text>
                    </view>
                    <view class="control-btns">
                        <button class="ctrl-btn ctrl-on" :class="deviceValueMap.BUZZER ? 'active' : ''" @click="setProperty('BUZZER', true)" :disabled="settingInProgress === 'BUZZER'">开启</button>
                        <button class="ctrl-btn ctrl-off" :class="!deviceValueMap.BUZZER ? 'active' : ''" @click="setProperty('BUZZER', false)" :disabled="settingInProgress === 'BUZZER'">关闭</button>
                    </view>
                </view>
            </view>

            <view class="footer">
```

```
                    <text class="footer-time">最后更新: {{lastUpdate}}</text>
                </view>
            </view>
        </view>
    </template>

    <script>
    const { createCommonToken } = require('@/key.js')

    // ========== OneNET 设备配置（来自微信小程序） ==========
    const PRODUCT_ID = '790G2UE7S4'
    const DEVICE_NAME = 'X1'
    const OFFLINE_THRESHOLD_MS = 15000

    // ========== 模拟天气 ==========
    const WEATHER_DATA = [
        { icon: '☀', temp: 28, desc: '晴天', city: '陕西-西安市' },
    ]

    // ========== 属性类型定义（来自用户提供的数据） ==========
    const DEVICE_PROPERTIES = [
        { identifier: 'ch4_concent',       type: 'float',    label: '甲烷浓度',       unit: '%' },
        { identifier: 'co',                type: 'float',    label: '一氧化碳',       unit: 'ppm' },
        { identifier: 'co2',               type: 'int32',    label: '二氧化碳',       unit: 'ppm' },
        { identifier: 'pm2_5',             type: 'int64',    label: 'PM2.5',          unit: 'μg/m³' },
        { identifier: 'env_humidity',      type: 'float',    label: '环境湿度',       unit: '%RH' },
        { identifier: 'env_temp',          type: 'float',    label: '环境温度',       unit: '°C' },
        { identifier: 'device_temp',       type: 'float',    label: '器件温度',       unit: '°C' },
        { identifier: 'output_current', type: 'float',    label: '输出电流',       unit: 'A' },
        { identifier: 'output_voltage', type: 'float',    label: '输出电压',       unit: 'V' },
        { identifier: 'runtime_hours',    type: 'int32',    label: '运行时长',       unit: 'h' },
        { identifier: 'health_score',      type: 'int32',    label: '健康评分',       unit: '分' },
        { identifier: 'LED',               type: 'bool',     label: 'LED 灯',         unit: '' },
        { identifier: 'BUZZER',            type: 'bool',     label: '蜂鸣器报警',    unit: '' },
    ]

    // ========== 值转换 ==========
    const valueConverter = (rawValue, type) => {
        switch (type) {
            case 'bool':     return rawValue === true || rawValue === 'true'
            case 'int32':
            case 'int64':    return parseInt(rawValue, 10) || 0
            case 'float':    return parseFloat(rawValue) || 0.0
            default:             return rawValue
```

```
    }
}

// ========== 颜色映射 ==========
const getDisplayColor = (identifier, value) => {
    if (typeof value === 'boolean') return '#000000'

    const rules = {
        env_humidity: [
            { min: 40, max: 60, color: '#07c160' },
            { min: 30, max: 40, color: '#999999' },
            { min: 60, max: 70, color: '#999999' },
            { max: 30, color: '#fa5151' },
            { min: 70, color: '#fa5151' }
        ],
        env_temp: [
            { min: 20, max: 26, color: '#07c160' },
            { min: 15, max: 20, color: '#999999' },
            { min: 26, max: 30, color: '#999999' },
            { max: 15, color: '#fa5151' },
            { min: 30, color: '#fa5151' }
        ],
        ch4_concent: [
            { min: 0, max: 5, color: '#07c160' },
            { min: 5, max: 7, color: '#999999' },
            { min: 7, max: 10, color: '#fa5151' }
        ],
        co: [
            { min: 0, max: 50, color: '#07c160' },
            { min: 50, max: 100, color: '#999999' },
            { min: 100, color: '#fa5151' }
        ],
        co2: [
            { min: 0, max: 1000, color: '#07c160' },
            { min: 1000, max: 2000, color: '#999999' },
            { min: 2000, color: '#fa5151' }
        ],
        pm2_5: [
            { min: 0, max: 50, color: '#07c160' },
            { min: 50, max: 100, color: '#999999' },
            { min: 100, color: '#fa5151' }
        ],
        health_score: [
            { min: 80, max: 100, color: '#07c160' },
```

```
            { min: 60, max: 80, color: '#999999' },
            { max: 60, color: '#fa5151' }
        ],
        output_current: [
            { min: 0.5, max: 3, color: '#07c160' },
            { min: 0, max: 0.5, color: '#999999' },
            { min: 3, max: 4, color: '#999999' },
            { min: 4, color: '#fa5151' }
        ],
        output_voltage: [
            { min: 12, max: 36, color: '#07c160' },
            { min: 0, max: 12, color: '#999999' },
            { min: 36, max: 48, color: '#999999' },
            { min: 48, color: '#fa5151' }
        ],
        device_temp: [
            { min: -20, max: 70, color: '#07c160' },
            { min: 70, max: 90, color: '#999999' },
            { min: 90, color: '#fa5151' }
        ],
        runtime_hours: [
            { min: 0, color: '#07c160' }
        ]
    }

    const list = rules[identifier]
    if (!list) return '#000000'
    for (const rule of list) {
        const minOk = rule.min === undefined || value >= rule.min
        const maxOk = rule.max === undefined || value <= rule.max
        if (minOk && maxOk) return rule.color
    }
    return '#000000'
}

export default {
    data() {
        return {
            // 设备数据
            deviceValueMap: {},
            deviceColorMap: {},
            deviceOnline: false,
            lastUpdate: '',
            lastUpdateTick: 0,
```

```
        // Token
        token: '',

        // 控制节流
        settingInProgress: null,

        // 定时器
        dataTimer: null,
        onlineCheckTimer: null,
        clockTimer: null,

        // 时钟天气
        clockTime: '',
        clockDate: '',
        weatherIcon: '☀',
        weatherTemp: 28,
        weatherDesc: '晴朗',
        weatherLoc: '陕西-西安市',
    }
},

onLoad() {
    this.initToken()
    this.initWeather()
    this.updateClock()
    this.clockTimer = setInterval(() => this.updateClock(), 1000)
},

onShow() {
    this.startDataUpdate()
    this.startOnlineCheck()
},

onHide() {
    this.stopDataUpdate()
    this.stopOnlineCheck()
},

onUnload() {
    this.stopDataUpdate()
    this.stopOnlineCheck()
    if (this.clockTimer) {
        clearInterval(this.clockTimer)
```

```
            this.clockTimer = null
        }
    },

    methods: {
        // ========== 工具 ==========
        formatVal(val) {
            if (val === undefined || val === null || val === '--') return '--'
            if (typeof val === 'number') {
                // 小数保留 2 位，整数不显示小数
                return Number.isInteger(val) ? val : val.toFixed(2)
            }
            return val
        },

        initWeather() {
            const w = WEATHER_DATA[Math.floor(Math.random() * WEATHER_DATA.length)]
            this.weatherIcon = w.icon
            this.weatherTemp = w.temp
            this.weatherDesc = w.desc
            this.weatherLoc = w.city
        },

        updateClock() {
            const d = new Date()
            const pad = n => n.toString().padStart(2, '0')
            this.clockTime = `${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`
            const weekdays = ['日', '一', '二', '三', '四', '五', '六']
            this.clockDate = `${d.getFullYear()} 年 ${pad(d.getMonth() + 1)} 月 ${pad(d.getDate())}日 周${weekdays[d.getDay()]}`
        },

        formatTime() {
            const d = new Date()
            const pad = n => n.toString().padStart(2, '0')
            return `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())} ${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`
        },

        // ========== Token ==========
        initToken() {
            try {
                const params = {
```

```
                author_key: 'c0a787ec7ab54a4b87c594006cc06dba',
                version: '2022-05-01',
                user_id: '490050',
            }
            this.token = createCommonToken(params)
        } catch (e) {
            console.error('Token 生成失败', e)
        }
    },

    // ========== 在线检测 ==========
    startOnlineCheck() {
        this.stopOnlineCheck()
        this.onlineCheckTimer = setInterval(() => {
            if (this.lastUpdateTick > 0 && (Date.now() - this.lastUpdateTick) > OFFLINE_THRESHOLD_MS) {
                this.deviceOnline = false
            }
        }, 3000)
    },

    stopOnlineCheck() {
        if (this.onlineCheckTimer) {
            clearInterval(this.onlineCheckTimer)
            this.onlineCheckTimer = null
        }
    },

    // ========== 数据刷新 ==========
    startDataUpdate() {
        this.stopDataUpdate()
        this.fetchDevData()
        this.dataTimer = setInterval(() => this.fetchDevData(), 3000)
    },

    stopDataUpdate() {
        if (this.dataTimer) {
            clearInterval(this.dataTimer)
            this.dataTimer = null
        }
    },

    fetchDevData() {
        if (!this.token) this.initToken()
```

```
        uni.request({
            url: 'https://iot-api.heclouds.com/thingmodel/query-device-property',
            method: 'GET',
            data: { product_id: PRODUCT_ID, device_name: DEVICE_NAME },
            header: { authorization: this.token },
            success: (res) => {
                if (res.data.code === 10403) {
                    this.initToken()
                    this.retryFetchDevData()
                    return
                }
                if (!res.data || !res.data.data) return
                this.processDevData(res.data.data)
                this.deviceOnline = true
                this.lastUpdateTick = Date.now()
                this.lastUpdate = this.formatTime()
            },
            fail: (err) => { console.log('获取数据失败', err) }
        })
    },

    retryFetchDevData() {
        if (!this.token) return
        uni.request({
            url: 'https://iot-api.heclouds.com/thingmodel/query-device-property',
            method: 'GET',
            data: { product_id: PRODUCT_ID, device_name: DEVICE_NAME },
            header: { authorization: this.token },
            success: (retryRes) => {
                if (!retryRes.data || !retryRes.data.data) return
                this.processDevData(retryRes.data.data)
                this.deviceOnline = true
                this.lastUpdateTick = Date.now()
                this.lastUpdate = this.formatTime()
            },
            fail: (err) => { console.log('重试获取数据失败', err) }
        })
    },

    processDevData(data) {
        const valueMap = { ...this.deviceValueMap }
        const colorMap = { ...this.deviceColorMap }
        let changed = false
```

```
            data.forEach(item => {
                // 查找类型定义
                const def = DEVICE_PROPERTIES.find(p => p.identifier === item.identifier)
                if (!def) return
                const val = valueConverter(item.value, def.type)
                if (valueMap[item.identifier] !== val) {
                    valueMap[item.identifier] = val
                    colorMap[item.identifier] = getDisplayColor(item.identifier, val)
                    changed = true
                }
            })

            if (changed) {
                this.deviceValueMap = valueMap
                this.deviceColorMap = colorMap
            }
        },

        // ========== 远程控制 ==========
        setProperty(field, value) {
            if (this.settingInProgress === field) return
            this.settingInProgress = field

            // 乐观更新
            const valueMap = { ...this.deviceValueMap, [field]: value }
            const colorMap = { ...this.deviceColorMap, [field]: getDisplayColor(field, value) }
            this.deviceValueMap = valueMap
            this.deviceColorMap = colorMap

            const params = {}
            params[field] = { value }

            uni.request({
                url: 'https://iot-api.heclouds.com/thingmodel/set-device-property',
                method: 'POST',
                data: { product_id: PRODUCT_ID, device_name: DEVICE_NAME, params },
                header: { authorization: this.token, 'Content-Type': 'application/json' },
                success: (res) => {
                    if (res.data && res.data.code === 0) {
                        uni.showToast({ title: '设置成功', icon: 'success', duration: 1500 })
                    } else {
                        uni.showToast({ title: res.data.msg || '设置失败', icon: 'none',
duration: 2000 })
                    }
```

```
                        },
                        fail: (err) => {
                            console.log(field + '=' + JSON.stringify(value) + ' fail', err)
                            if (err.data && err.data.code === 10403) this.initToken()
                            uni.showToast({ title: '网络错误，设置失败', icon: 'none', duration:
2000 })
                        },
                        complete: () => { this.settingInProgress = null }
                    })
            },
        }
}
</script>

<style>
/* ========== 全局 ========== */
.page {
    background: #f5f6fa;
    min-height: 100vh;
}
.wrap {
    padding: 20rpx 30rpx 40rpx;
    overflow-y: auto;
    box-sizing: border-box;
}

/* ========== 天气时间 ========== */
.weather-section {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: linear-gradient(135deg, #74b9ff, #0984e3);
    border-radius: 28rpx;
    padding: 24rpx 30rpx;
    margin-bottom: 24rpx;
    color: #fff;
}
.weather-left { display: flex; align-items: center; gap: 16rpx; }
.weather-icon { font-size: 64rpx; }
.weather-temp { font-size: 44rpx; font-weight: bold; display: block; }
.weather-desc { font-size: 24rpx; opacity: 0.9; }
.weather-right { text-align: right; }
.weather-time { font-size: 48rpx; font-weight: bold; display: block; letter-spacing: 2rpx; }
.weather-date { font-size: 22rpx; opacity: 0.85; display: block; margin-top: 4rpx; }
```

```
.weather-location { font-size: 22rpx; opacity: 0.75; display: block; margin-top: 4rpx; }

/* ========== 设备信息栏 ========== */
.profile-section {
    display: flex;
    align-items: center;
    background: linear-gradient(135deg, #667eea, #764ba2);
    border-radius: 28rpx;
    padding: 28rpx 30rpx;
    margin-bottom: 24rpx;
    color: #fff;
}
.profile-avatar { font-size: 64rpx; margin-right: 24rpx; }
.profile-info { flex: 1; }
.profile-name { font-size: 32rpx; font-weight: bold; display: block; }
.profile-desc { font-size: 22rpx; opacity: 0.8; margin-top: 4rpx; display: block; }
.status-badge { font-size: 22rpx; padding: 8rpx 20rpx; border-radius: 30rpx; font-weight: bold; }
.status-badge.online { background: rgba(39, 174, 96, 0.3); color: #fff; }
.status-badge.offline { background: rgba(149, 165, 166, 0.3); color: #ddd; }

/* ========== 标题 ========== */
.section-title {
    font-size: 30rpx;
    font-weight: bold;
    color: #2c3e50;
    margin-bottom: 20rpx;
}

/* ========== 数据网格 ========== */
.data-container {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20rpx;
    margin-bottom: 32rpx;
}

.data-item {
    background: #fff;
    border-radius: 24rpx;
    padding: 24rpx 20rpx;
    display: flex;
    align-items: center;
    box-shadow: 0 4rpx 20rpx rgba(0,0,0,0.06);
}
```

```
.data-item-invisible {
	visibility: hidden;
}

.data-left {
	width: 80rpx;
	height: 80rpx;
	border-radius: 20rpx;
	display: flex;
	align-items: center;
	justify-content: center;
	margin-right: 16rpx;
	background: #f0f4ff;
	flex-shrink: 0;
}

.data-emoji {
	font-size: 40rpx;
}

.data-right {
	flex: 1;
	min-width: 0;
}

.data-label {
	font-size: 24rpx;
	color: #636e72;
	display: block;
	margin-bottom: 4rpx;
}

.data-value {
	font-size: 40rpx;
	font-weight: bold;
	color: #2d3436;
	line-height: 1.2;
}

.data-unit {
	font-size: 22rpx;
	font-weight: normal;
	color: #999;
```

```
}

/* ========== 远程控制 ========== */
.control-section {
	margin-bottom: 24rpx;
}

.control-card {
	display: flex;
	align-items: center;
	justify-content: space-between;
	background: #fff;
	border-radius: 24rpx;
	padding: 24rpx 30rpx;
	margin-bottom: 16rpx;
	box-shadow: 0 4rpx 20rpx rgba(0,0,0,0.06);
}

.control-info {
	display: flex;
	align-items: center;
	gap: 16rpx;
	flex: 1;
}

.control-icon { font-size: 40rpx; }
.control-label { font-size: 28rpx; font-weight: bold; color: #2d3436; }

.control-btns {
	display: flex;
	gap: 12rpx;
}

.ctrl-btn {
	font-size: 24rpx;
	padding: 12rpx 28rpx;
	border-radius: 30rpx;
	border: 2rpx solid #b2bec3;
	background: #fff;
	color: #636e72;
	font-weight: bold;
	line-height: initial;
	text-align: center;
}
```

```
.ctrl-btn.active {
	border-color: transparent;
	color: #fff;
}

.ctrl-on.active { background: #27ae60; }
.ctrl-off.active { background: #e74c3c; }

.ctrl-btn[disabled] {
	opacity: 0.5;
}

/* ==========  底部  ========== */
.footer {
	text-align: center;
	padding: 20rpx 0 40rpx;
}

.footer-time {
	font-size: 22rpx;
	color: #b2bec3;
}
</style>
<script>
	export default {
		onLaunch: function() {
			console.log('App Launch')
		},
		onShow: function() {
			console.log('App Show')
		},
		onHide: function() {
			console.log('App Hide')
		}
	}
</script>

<style>
	/*每个页面公共 css */
</style>
<script>
	export default {
		onLaunch: function() {
```

```
            console.log('App Launch')
        },
        onShow: function() {
            console.log('App Show')
        },
        onHide: function() {
            console.log('App Hide')
        }
    }
</script>

<style>
    /*每个页面公共 css */
</style>
<script>
    export default {
        onLaunch: function() {
            console.log('App Launch')
        },
        onShow: function() {
            console.log('App Show')
        },
        onHide: function() {
            console.log('App Hide')
        }
    }
</script>

<style>
    /*每个页面公共 css */
</style>
import App from './App'

// #ifndef VUE3
import Vue from 'vue'
import './uni.promisify.adaptor'
Vue.config.productionTip = false
App.mpType = 'app'
const app = new Vue({
  ...App
})
app.$mount()
// #endif
```

```
// #ifdef VUE3
import { createSSRApp } from 'vue'
export function createApp() {
    const app = createSSRApp(App)
    return {
        app
    }
}
// #endif
```